要件定義書 — 交通事故監控 API 平台

> Back-End Candidate Evaluation 対応ドキュメント

---

01 項目概要

項目	内容
システム名	Traffic Incident Monitoring API Platform
目的	AI ビデオ解析の検出イベントを受信し、調度センターへ即時配信する
対象ユーザー	高速道路オペレーションセンタースタッフ
提出形式	Git リポジトリ + README + AI 対話ログ
技術制約	なし（言語・フレームワーク自由）

> **設計思想**：Speed matters. SSE によるリアルタイムプッシュで検出から表示までのレイテンシを最小化する。

---

02 機能要件

必須 Required

ID	エンドポイント	内容
F-01	`POST /events`	AI 検出システムからイベント受信。Pydantic バリデーション、受信後即座に SSE ブロードキャスト
F-02	`GET /events`	一覧取得。filter（severity / event_type / status）、sort（detected_at / severity）、ページネーション対応
F-03	`GET /events/{id}`	単一イベント取得。存在しない場合 404

推奨 Nice to Have

ID	機能	内容
F-04	`GET /events/stream`	SSE 長期接続。新イベント POST のたびに全 Dashboard へ即時配信。「Speed matters」への直接的な回答

ID	機能	内容
F-05	`PATCH /events/{id}/status`	ステータス更新（NEW → ACKNOWLEDGED → DISPATCHED → RESOLVED）。更新後も SSE ブロードキャストを発火し、他オペレーターの画面をリアルタイム同期する
F-06	Docker Compose	`docker compose up` 一コマンドで全環境起動
F-07	Seed Script	`python scripts/seed.py` でランダムイベントを自動 POST し続ける
F-08	Dashboard UI	静的 HTML + vanilla JS。SSE でリアルタイム更新
F-09	YOLO 連携	YOLOv8 で動画解析 → `POST /events` へ自動送信（自選加分）

03 API エンドポイント規格

Method	Path	用途	Status
POST	`/events`	イベント受信	201 Created
GET	`/events`	一覧取得	200 OK
GET	`/events/{id}`	単一取得	200 / 404
PATCH	`/events/{id}/status`	ステータス更新	200 OK
GET	`/events/stream`	SSE 長期接続	200 text/event-stream

Query Parameters — GET /events

パラメータ	型	例	説明
`severity`	string	`HIGH,CRITICAL`	カンマ区切りで複数指定可
`event_type`	string	`DEBRIS`	単一指定
`status`	string	`NEW`	単一指定
`sort_by`	string	`detected_at`	`detected_at` / `severity`
`order`	string	`desc`	`asc` / `desc`（デフォルト `desc`）

パラメータ	型	例	説明
`limit`	int	`20`	デフォルト 20、最大 100
`offset`	int	`0`	デフォルト 0

統一レスポンス形式

```json
{
  "success": true,
  "data": [...],
  "total": 42,
  "limit": 20,
  "offset": 0
}
```

---

04 データモデル

incident_events テーブル

フィールド	型	必須	説明
`id`	UUID	✅	主キー、自動生成
`event_type`	ENUM	✅	STOPPED_VEHICLE / DEBRIS / CONGESTION / WRONG_WAY / PEDESTRIAN / UNKNOWN
`severity`	ENUM	✅	LOW / MEDIUM / HIGH / CRITICAL
`status`	ENUM	自動	NEW / ACKNOWLEDGED / DISPATCHED / RESOLVED（デフォルト NEW）
`description`	TEXT	—	AI が生成したイベント説明
`confidence`	FLOAT	✅	AI 検出信頼度 0.0〜1.0
`camera_id`	VARCHAR	✅	カメラ ID
`highway_id`	VARCHAR	—	路線番号（例：E1、C2）
`latitude`	FLOAT	✅	緯度
`longitude`	FLOAT	✅	経度

フィールド	型	必須	説明
`image_url`	VARCHAR	—	検出時スナップショット URL
`detected_at`	TIMESTAMP	✅	AI が検出した時刻（送信側が設定）
`ingested_at`	TIMESTAMP	自動	API が受信した時刻（遅延計測用）
`updated_at`	TIMESTAMP	自動	最終更新時刻

> `detected_at` と `ingested_at` の差分 = 検出遅延。Dashboard に表示することでシステムの健全性を可視化できる。

---

05 非機能要件

項目	要件	実現方法
API ドキュメント	自動生成	FastAPI 内蔵 Swagger UI（`/docs`）
バリデーション	全フィールド検証	Pydantic v2
エラーハンドリング	400/404/422/500 明示	FastAPI exception handler
環境設定	`.env` 管理	python-dotenv
コード構造	分層アーキテクチャ	router / service / repository / schema
認証	**Scope 外** ※	—
テスト	主要エンドポイント	pytest + httpx

> ※ 認証・認可は本テストのスコープ外。README に「本番環境では API Key / JWT Bearer Token が必要」と明記する。

---

06 YOLO データセット選定

場景	主データセット	形式	戦略	Fine-tune
路肩停車	BDD100K + MIO-TCD	JSON → YOLO txt 変換	車両検出 + ByteTrack + ROI 停留時間判定	必須

場景	主データセット	形式	戦略	Fine-tune
路面破片	RAOD（UnicomAI）	mask → bbox 変換	YOLOv8-seg、合成 copy-paste 増強	必須（強め）
異常渋滞	TRANCOS + Mendeley congestion	YOLO txt	密度カウント + 速度推定 + 閾値ルール	必須

モデル選定

場景	モデル	入力解像度	備考
路肩停車	yolov8m.pt	640×640	ByteTrack 追跡と併用
路面破片	yolov8m-seg.pt	1280×1280	小目標のため高解像度
異常渋滞	yolov8s.pt	640×640	スピード優先、カウンティング重視

---

07 ディレクトリ構成

```
traffic-incident-api/
├── app/
|    ├── main.py
|    ├── database.py
|    ├── models/
|    |    └── incident.py          # SQLAlchemy ORM
|    ├── schemas/
|    |    └── incident.py          # Pydantic schema
|    ├── routers/
|    |    ├── incidents.py          # POST / GET /events
|    |    └── stream.py            # SSE /events/stream
|    └── services/
|         ├── incident_service.py  # ビジネスロジック
|         └── sse_manager.py       # SSE ブロードキャスト管理
├── scripts/
|    └── seed.py                   # 仮想イベント生成
├── yolo/
|    └── detector.py               # YOLO 連携（自選）
├── ui/
|    └── index.html                # ダッシュボード
├── tests/
|    └── test_incidents.py
├── docker-compose.yml
├── Dockerfile
```

```
├── requirements.txt
└── README.md
```

---

08 実装ステップ

Phase 1 — Backend API（Day 1-2）

Step 1：プロジェクト初期化

- ディレクトリ構成を作成
- `requirements.txt` に依存関係を追加

```
fastapi uvicorn sqlalchemy pydantic python-dotenv httpx pytest
```

- `app/core/config.py` に環境変数設定

Step 2：データモデル定義

- `app/models/incident.py` に SQLAlchemy ORM モデルを定義
- ENUM 型（EventType / Severity / Status）を定義
- `app/database.py` で SQLite 接続と `Base.metadata.create_all()` を設定

```python
class IncidentEvent(Base):
    __tablename__ = "incident_events"
    id          = Column(String, primary_key=True, default=lambda: str(uuid4()))
    event_type  = Column(Enum(EventType), nullable=False)
    severity    = Column(Enum(Severity),  nullable=False)
    status      = Column(Enum(Status),    default="NEW")
    confidence  = Column(Float,           nullable=False)
    camera_id   = Column(String,          nullable=False)
    latitude    = Column(Float,           nullable=False)
    longitude   = Column(Float,           nullable=False)
    description = Column(Text)
    highway_id  = Column(String)
    image_url   = Column(String)
    detected_at = Column(DateTime, nullable=False)
    ingested_at = Column(DateTime, default=datetime.utcnow)
    updated_at  = Column(DateTime, onupdate=datetime.utcnow)
```

Step 3：Pydantic Schema 定義

- `IncidentCreate`：POST 受信用
- `IncidentResponse`：GET レスポンス用（id / status / ingested_at 追加）
- `APIResponse`：統一レスポンスラッパー

Step 4：POST /events 実装

- router → service → repository の分層構造で実装
- 保存成功後、SSE Manager へブロードキャスト通知
- 201 レスポンスに作成イベントを含めて返す

Step 4b：PATCH /events/{id}/status 実装

- ステータス更新後、必ず SSE ブロードキャストを発火する
- `type: "STATUS_UPDATE"` を付けて送ることで前端側が「新規追加」と「ステータス変更」を区別できる

```python
# incident_service.py
async def update_status(id, new_status, db, sse_manager):
    event = db.query(IncidentEvent).filter_by(id=id).first()
    event.status = new_status
    db.commit()
    await sse_manager.broadcast({        # ← 忘れると重複派工が発生する
        "type": "STATUS_UPDATE",
        "id": id,
        "status": new_status
    })
    return event
```

Step 5：GET /events 実装

- Query パラメータを SQLAlchemy の filter / order_by に変換
- total カウントを別クエリで取得
- `GET /events/{id}` も合わせて実装

Step 6：SSE エンドポイント実装

```python
```

```python
class SSEManager:
    def __init__(self):
        self.connections: list[asyncio.Queue] = []

    async def broadcast(self, data: dict):
        for q in self.connections:
            await q.put(data)

    async def stream(self, queue: asyncio.Queue):
        while True:
            data = await queue.get()
            yield f"data: {json.dumps(data)}\n\n"
```

Step 7：テスト作成

- pytest + httpx.AsyncClient で主要エンドポイントをカバー
- バリデーションエラー（422）、Not Found（404）も確認

Phase 2 — YOLO 連携（Day 3-4）

> ⚠️ **リスク管理：Phase 1 / 3 / 4 が 100% 完成してから着手すること。** 時間が足りない場合は Fine-tune を省略し、`yolov8n.pt`（COCO 事前学習）でデモを動かすだけでよい。 面試官が評価するのはモデル精度ではなく「端到端のシステム統合能力」。 README に「時間があれば RAOD fine-tune を実施する」と明記すれば十分。

Step 1：環境セットアップ

```bash
pip install ultralytics
yolo predict model=yolov8n.pt source=test.mp4  # 動作確認
```

Step 2：最小デモ（時間が足りない場合はここまででよい）

- `yolov8n.pt` で任意の高速道路動画を解析
- car クラスを検出 → `STOPPED_VEHICLE` イベントとして POST
- Fine-tune なし、端到端の連携ができれば OK

Step 3：場景 B（路面碎片）— 時間があれば

- RAOD の mask を bbox へ変換するスクリプト作成
- YOLOv8m-seg で fine-tune（最も純粋な検出タスク、実装が一番シンプル）

```bash
```

```
yolo train model=yolov8m-seg.pt data=raod.yaml epochs=50 imgsz=1280
```

Step 4：場景 A（路肩停車）— 時間があれば

- BDD100K / MIO-TCD で車両検出器を fine-tune
- ByteTrack で追跡、路肩 ROI を手動定義
- ROI 内に N 秒停留 → STOPPED_VEHICLE イベント発火

Step 5：場景 C（異常拥堵）— 時間があれば

- TRANCOS / Mendeley で車両カウンター学習
- ROI 内の密度 + 平均速度を計算
- 高密度 + 低速度が閾値超過 → CONGESTION イベント発火

Step 6：API 統合

```python
import httpx

payload = {
    "event_type": "DEBRIS",
    "severity": "HIGH",
    "confidence": float(conf),
    "camera_id": "CAM-001",
    "latitude": 35.123,
    "longitude": 137.456,
    "detected_at": datetime.utcnow().isoformat(),
}
httpx.post("http://localhost:8000/events", json=payload)
```

Phase 3 — インフラ整備（Day 4）

Step 1：Dockerfile

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 2：docker-compose.yml

```yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    volumes: ["./data:/app/data"]   # SQLite 永続化
    environment:
      DATABASE_URL: sqlite:///./data/incidents.db
  seed:
    build: .
    command: python scripts/seed.py
    depends_on: [api]
```

Step 3：Seed Script

- 3 種類のイベントをランダムに生成
- 2〜5 秒ごとに POST し続ける
- 日本の高速道路座標（東名・中央道付近）を使用

Phase 4 — Dashboard UI（Day 5）

Step 1：HTML 構成

- フィルターバー（severity / event_type / status）
- イベントリスト（detected_at / severity / location / description）
- severity 別カラーコーディング

Step 2：SSE 接続（重複排除付き）

初期ロード時と SSE プッシュの間にタイムラグが生じると、同一イベントが二重表示される。`Set` で ID を管理して防ぐ。

```javascript
```

```
const renderedIds = new Set();

function renderEvent(event) {
    if (renderedIds.has(event.id)) return;  // 重複をスキップ
    renderedIds.add(event.id);
    prependEventToList(event);
}

function updateEventStatus(id, status) {
    const card = document.querySelector(`[data-id="${id}"]`);
    if (card) card.dataset.status = status;  // カード更新のみ、新規追加しない
}

// 初期ロード
fetch("/events").then(r => r.json()).then(r => r.data.forEach(renderEvent));

// SSE プッシュ
const es = new EventSource("/events/stream");
es.onmessage = (e) => {
    const msg = JSON.parse(e.data);
    if (msg.type === "STATUS_UPDATE") {
        updateEventStatus(msg.id, msg.status);  // 更新のみ
    } else {
        renderEvent(msg);                       // 重複排除して追加
        if (msg.severity === "CRITICAL") playAlert();
    }
};
```

Step 3：遅延表示（アピールポイント）

- `ingested_at - detected_at` を「検出から X 秒後に受信」として各イベントに表示

---

09 スコープ外（README に明記）

項目	本番対応方針
認証・認可	API Key / JWT Bearer
レート制限	nginx / API Gateway
画像ストレージ	S3 / CloudFront
本番 DB	PostgreSQL
HTTPS / TLS	nginx reverse proxy

項目	本番対応方針
YOLO モデル精度保証	本番用データ収集 + fine-tune が必要